

Résumé détaillé — Chapitres 1 & 2

Sécurité des applications (2025/2026)

Synthèse pédagogique (module SEC APP)

5 avril 2026

Table des matières

1	Chapitre 1 — Bases de la sécurité	2
1.1	Contexte du module et outils TP	2
1.2	Pourquoi la sécurité applicative ?	2
1.3	Surface d'attaque d'une application web	2
1.4	Niveaux de sécurité	2
1.5	Triade CIA et modèle AAA	3
1.6	Menace, vulnérabilité, attaque, risque	3
1.7	Taxonomie des attaques	3
1.8	Vecteurs d'attaque fréquents (web)	3
1.9	Cyber Kill Chain (Lockheed Martin)	3
1.10	OWASP Top 10 — 2021 (synthèse)	4
1.11	Huit principes de Salter et Schroeder (1975)	4
1.12	SDL et DevSecOps	4
1.13	Bonnes pratiques de codage (rappel)	4
2	Chapitre 2 — Sécurité du client web	5
2.1	Objectifs pédagogiques	5
2.2	Architecture du navigateur	5
2.3	DOM et Same-Origin Policy (SOP)	5
2.4	API JavaScript à risque	5
2.5	Cross-Site Scripting (XSS)	5
2.6	Cross-Site Request Forgery (CSRF)	5
2.7	Injection SQL (SQLi)	5
2.8	IDOR (Insecure Direct Object Reference)	6
2.9	En-têtes HTTP de sécurité (rappel)	6
2.10	CORS	6
2.11	Clickjacking	6
2.12	Synthèse (matrice cours)	6
3	Consignes globales de révision	6
4	Conseils méthodologiques	6
5	Informations complémentaires	7

Introduction

Ce document résume les **Chapitre 1 : Bases de la sécurité** et **Chapitre 2 : Sécurité du client web** du cours *Sécurité des applications*. Il ajoute des **consignes de travail**, des **conseils méthodologiques** et des **informations complémentaires** pour la révision et la mise en pratique (TP).

1 Chapitre 1 — Bases de la sécurité

1.1 Contexte du module et outils TP

Le module couvre ensuite la sécurité réseau, l'architecture des serveurs web, etc. Les **liens utiles** du cours incluent notamment :

- ASVS OWASP : <https://owasp.org/www-project-application-security-verification-standard/>
- TLS côté serveur (Mozilla) : https://wiki.mozilla.org/Security/Server_Side_TLS
- HTTPS / handshake TLS (Cloudflare)
- OAuth 2 simplifié, hardening Nginx, *Logging Cheat Sheet* OWASP

Environnement TP (à installer) : Wireshark, Nmap, OWASP ZAP, Docker ; VMs Ubuntu, Kali, Metasploitable 2, DVWA ; option Apache + ModSecurity.

Outils clés : Docker Desktop, OWASP Juice Shop (cible vulnérable), Burp Suite Community, Firefox + DevTools, WebGoat, SQLMap, Python 3 + **bcrypt**, et outils en ligne (CrackStation, jwt.io, webhook.site).

Consigne : Installez l'environnement *avant* les séances : une VM attaquante (Kali) et une cible (Ubuntu + Docker / Juice Shop). Vérifiez que Burp intercepte bien le trafic HTTP(S) avec le certificat importé dans le navigateur.

Conseil : Notez dans un carnet les ports par défaut cités en cours : Juice Shop (3000), WebGoat (8080), WebWolf (9090), DVWA (8081) — cela accélère le débogage des TP.

1.2 Pourquoi la sécurité applicative ?

Indicateurs (ordre de grandeur, sources du cours) : coût moyen d'une violation $\approx 4,88$ M USD (IBM 2024) ; détection d'intrusion ≈ 194 jours ; forte hausse des cyberattaques ; $\approx 43\%$ des attaques ciblent le web (Verizon DBIR) ; part importante des failles liées au code applicatif (statistiques OWASP/NIST).

1.3 Surface d'attaque d'une application web

- **Présentation** : formulaires, URLs, GET/POST, uploads, cookies, en-têtes HTTP.
- **Logique métier** : APIs, autorisation, sessions, règles métier.
- **Données** : SQL/NoSQL, ORM, configuration, logs.
- **Infrastructure** : serveurs, fichiers, tiers, dépendances.
- **Authentification** : login, reset MDP, 2FA, tokens, SSO.

1.4 Niveaux de sécurité

Réseau

Périmètre réseau ; pare-feu, NIPS, IDS, VPN.

Système

OS et services ; antivirus, HIPS/HIDS, patches.

Applicative

Une application précise ; correctifs, WAF, IPS applicatif, validation des entrées.

Extensions évoquées : embarqué (IoT), Big Data, IA/ML.

1.5 Triade CIA et modèle AAA

CIA (référence depuis les années 1970) :

Confidentialité

Accès aux données réservé aux entités autorisées. Menaces : écoute, vol de credentials, exposition en clair. Mesures : chiffrement (AES, TLS), auth forte, RBAC, moindre privilège. Ex. : dossier médical visible seulement par le médecin traitant.

Intégrité

Exactitude et absence de modification non autorisée. Menaces : falsification, injection SQL, MitM. Mesures : signatures, hachage (ex. SHA-256), journaux, validation des entrées. Ex. : montant de virement non altéré en transit.

Disponibilité

Services utilisables quand nécessaire. Menaces : DDoS, pannes, ransomware, mauvaise config. Mesures : redondance, sauvegardes, PRA/PCA, load balancing. Ex. : dossiers patients accessibles 24h/24.

AAA complète CIA pour la gestion des accès :

- **Authentication** — « Qui êtes-vous ? » (MFA, certificats, etc.) ; faiblesse fréquente des incidents.
- **Authorization** — « Que pouvez-vous faire ? » (RBAC, ABAC, ACL) ; moindre privilège.
- **Accounting** — « Qu'avez-vous fait ? » (logs, SIEM, audit).

Propriétés complémentaires (ISO 27001, NIST) : non-répudiation, authenticité, fiabilité, résilience, vie privée (RGPD, loi algérienne 18-07 citée en cours).

1.6 Menace, vulnérabilité, attaque, risque

- **Menace** : acteur ou événement pouvant causer un dommage (potentiel).
- **Vulnérabilité** : faiblesse exploitable (interne au système).
- **Attaque** : exploitation concrète d'une vulnérabilité par une menace.
- **Risque** : combinaison probabilité $\times$ impact ; guide les priorités.
- **Contre-mesure** : réduit la probabilité ou l'impact.
- **Actif** : ce qui a de la valeur à protéger.

1.7 Taxonomie des attaques

- **Passives** : observation (sniffing, OSINT) — discrètes.
- **Actives** : modification, destruction — plus visibles.
- **Origine** : externes, internes (souvent dangereuses), par rebond (pivot).

1.8 Vecteurs d'attaque fréquents (web)

Injections (SQL, NoSQL, OS, LDAP) ; XSS ; authentification faible ; phishing / ingénierie sociale ; MitM ; supply chain ; DoS/DDoS ; exploitation de CVE.

1.9 Cyber Kill Chain (Lockheed Martin)

Phases : reconnaissance $\rightarrow$ armement $\rightarrow$ livraison $\rightarrow$ exploitation $\rightarrow$ installation $\rightarrow$ C&C $\rightarrow$ actions sur objectifs — avec exemples de détection/blocage par phase (patch, WAF, SIEM, DLP, etc.).

1.10 OWASP Top 10 — 2021 (synthèse)

OWASP : fondation à but non lucratif ; Top 10 mis à jour tous les 3–4 ans. La version 2021 met l'accent sur la **conception non sécurisée** (A04) et la **chaîne d'approvisionnement** (A08).

ID	Famille (résumé)	Protection clé
A01	Contrôle d'accès défaillant	Vérifier autorisations serveur
A02	Défaillances cryptographiques	HTTPS, bcrypt, AES-256
A03	Injection	Requêtes préparées, validation
A04	Conception non sécurisée	Threat modeling, SDL
A05	Mauvaise configuration	Hardening, pas de défauts
A06	Composants vulnérables	SBOM, scan CVE
A07	Authentification défaillante	MFA, politique MDP
A08	Intégrité logicielle/données	Signatures, CI/CD sûr
A09	Journalisation insuffisante	SIEM, logs complets
A10	SSRF	Whitelist d'URL, firewall

1.11 Huit principes de Salter et Schroeder (1975)

1. **Moindre privilège** — droits minimaux nécessaires.
2. **Défense en profondeur** — plusieurs couches indépendantes.
3. **Échec sécurisé** (*fail secure*) — en cas de doute, refuser l'accès.
4. **Économie des mécanismes** — simplicité auditable.
5. **Médiation complète** — révéifier chaque accès aux ressources.
6. **Conception ouverte** — pas de sécurité par l'obscurité ; secret = clés (Kerckhoffs).
7. **Séparation des privilèges** — opérations critiques multi-conditions.
8. **Mécanismes peu intrusifs** — limiter le partage (fuite latérale).

1.12 SDL et DevSecOps

SDL (Secure Development Lifecycle) : formation → exigences (menaces STRIDE, PIA) → conception → implémentation (SAST, normes) → vérification (DAST, pentest, ZAP, Burp) → déploiement (hardening, secrets) → maintenance (SIEM, CVE).

DevSecOps : SAST (SonarQube, Semgrep, Bandit...), DAST (ZAP, Burp), SCA (Snyk, Dependabot), détection de secrets (GitLeaks, TruffleHog), sécurité IaC (Checkov, tfsec), sécurité conteneurs (Trivy, Clair).

1.13 Bonnes pratiques de codage (rappel)

Validation **côté serveur** et listes blanches ; pas de secrets dans le code ; messages d'erreur génériques côté client ; sessions robustes (tokens aléatoires, invalidation) ; dépendances à jour et audit ; revues de code avec checklist sécurité.

Consigne : Pour chaque principe CIA/AAA ou Salter–Schroeder, préparez *un* exemple concret tiré d'une appli web (e-commerce, forum, banque) — c'est la forme de question la plus fréquente à l'oral ou à l'écrit.

Info + : Le **Threat Modeling** (STRIDE, diagrammes de flux de données) est le passage obligé entre « idée fonctionnelle » et architecture qui résiste aux abus métier (A04). OWASP propose des guides et modèles gratuits.

2 Chapitre 2 — Sécurité du client web

2.1 Objectifs pédagogiques

Comprendre le navigateur comme surface d'attaque ; maîtriser XSS (réfléchi, stocké, DOM), CSRF, IDOR ; relier aux en-têtes HTTP, CORS et clickjacking ; appliquer CSP, tokens CSRF, validation/encodage.

2.2 Architecture du navigateur

Moteur de rendu (DOM/CSSOM), moteur JS (V8, SpiderMonkey...), pile réseau (HTTP/S, cache, cookies, certificats), UI, stockage (cookies, `localStorage`, `IndexedDB`), **sandbox** par onglet.

2.3 DOM et Same-Origin Policy (SOP)

Le **DOM** est l'arbre d'objets manipulable par JavaScript — point d'entrée majeur des XSS.

SOP : un script d'origine A ne lit pas les données de l'origine B. **Origine** = schéma + hôte + port. Exemples du cours : même site/port OK ; changement de protocole, domaine, port ou sous-domaine $\Rightarrow$ blocage.

2.4 API JavaScript à risque

`innerHTML`, `document.write`, `eval`, `outerHTML` — exécutent ou interprètent des contenus contrôlables par un attaquant si alimentés par des entrées non sûres.

2.5 Cross-Site Scripting (XSS)

Définition : injection de JS exécuté par le navigateur de la victime dans le contexte du site légitime.

Impacts : vol de session (`document.cookie`), phishing de page, keylogging, défacement (surtout XSS stocké).

Réfléchi : payload dans l'URL ; non persistant ; flux attaquant $\rightarrow$ lien $\rightarrow$ victime $\rightarrow$ réponse reflétant le payload.

Stocké : payload en base (ex. commentaire) ; affecte tous les visiteurs ; très impactant.

DOM-based : le serveur ne voit pas le payload ; injection via JS client (ex. `innerHTML` avec paramètre URL) ; corriger avec `textContent` ou équivalent sûr.

CSP : en-tête listant les sources autorisées pour scripts, styles, images... ; limite l'exécution des scripts inline non autorisés. Directives typiques : `default-src 'self'`, `script-src`, `report-uri` / reporting moderne.

2.6 Cross-Site Request Forgery (CSRF)

Le serveur fait confiance au cookie de session : un site tiers peut déclencher une requête authentifiée.

Protections : token CSRF (non lisible cross-origin grâce à la SOP) ; cookies `SameSite=Strict` ou `Lax` ; vérification `Origin/Referer` (attention aux en-têtes parfois absents).

2.7 Injection SQL (SQLi)

Concaténation de requêtes = danger. **Requêtes préparées** / paramètres liés : l'entrée ne modifie pas la structure SQL. Types évoqués : in-band/UNION, boolean blind, time-based, error-based, OOB. Moindre privilège en base.

2.8 IDOR (Insecure Direct Object Reference)

Accès à des objets par ID prévisible sans vérifier la **propriété** ou le **rôle** : IDOR horizontal (données d'un pair) vs vertical (fonctions admin). Correction : toujours filtrer par `userId` (ou équivalent) côté serveur.

2.9 En-têtes HTTP de sécurité (rappel)

- CSP (Content-Security-Policy)
- HSTS (Strict-Transport-Security)
- Anti-clickjacking : `X-Frame-Options: DENY` ou `CSP frame-ancestors`
- `X-Content-Type-Options: nosniff`
- `Referrer-Policy`, `Permissions-Policy`
- Supprimer `X-Powered-By`

2.10 CORS

Mécanisme d'assouplissement contrôlé de la SOP.

Configuration dangereuse : origine `*` avec envoi des identifiants (`Allow-Credentials: true`). Sécurisé : liste blanche d'origines. Préflight `OPTIONS` pour requêtes « non simples ».

2.11 Clickjacking

Iframe transparente superposée pour détourner les clics. Protection : `X-Frame-Options: DENY` ou `CSP frame-ancestors 'none'`.

2.12 Synthèse (matrice cours)

Le cours relie chaque faille à un vecteur, un impact OWASP et une contre-mesure principale (voir tableau récapitulatif du support).

Consigne : TP1 (SOP + CORS) : reproduisez le blocage dans la console réseau, puis documentez *en une page* pourquoi la réponse préflight ou l'absence d'en-tête CORS empêche la lecture cross-origin — schéma requête/réponse obligatoire.

Conseil : Pour XSS : entraînez-vous à distinguer les trois types sans lire le code serveur (question : « le payload est-il dans la réponse HTML ? dans la base ? uniquement dans l'URL et le DOM ? »).

Info + : Les cookies de session doivent idéalement combiner `HttpOnly` (réduit vol par XSS), `Secure` (HTTPS seulement), `SameSite`, et durée de vie limitée — le cours insiste sur la session comme clé du compte.

3 Consignes globales de révision

1. **Maîtriser les définitions** : menace vs vulnérabilité vs risque ; origine ; CIA/AAA.
2. **Savoir placer** chaque attaque du ch. 2 sur la surface d'attaque (présentation, logique, données).
3. **Associer systématiquement** : vulnérabilité → OWASP Top 10 → contre-mesure principale.
4. **Pratiquer** sur Juice Shop / DVWA : une vulnérabilité comprise vaut mieux que dix slides relues.
5. **Rédiger** des fiches « une page » par famille : XSS, CSRF, SQLi, IDOR, CORS, headers.

4 Conseils méthodologiques

- Tracer les données utilisateur du point d'entrée jusqu'à la sortie (HTML, SQL, shell, URL) — c'est le fil conducteur des injections.
- Ne jamais se fier au contrôle *uniquement* côté client : tout peut être modifié (Burp).
- En examen, citer **le mécanisme** (SOP, CSP, prepared statement) avant l'outil commercial.
- Maintenir une liste personnelle de **CVE** ou incidents (SolarWinds, Heartbleed) comme illustrations de supply chain ou de patch urgent.

5 Informations complémentaires

- **OWASP Cheat Sheet Series** : <https://cheatsheetseries.owasp.org/> (XSS, CSRF, CSP, Logging...).
- **Mozilla Observatory** et **securityheaders.com** : vérification rapide des en-têtes.
- **CVE** : <https://cve.mitre.org/> pour la veille sur les composants.
- **ASVS** : grille d'exigences pour auditer une application de façon structurée.
- Normalisation des configurations TLS : guides Mozilla SSL Configuration Generator.